

EXPLORER UNIVERSE PRODUCTION SYSTEM

MVP Implementation Blueprint v1.1

STATUS:	VERSION:	ROLE:	TARGET ENGINE:
APPROVED FOR BUILD	1.1 (Dynamic Expansion)	Lead Technical Architect	AI Agents & Developers

EXECUTIVE SUMMARY & STRATEGY

The objective of the **Minimum Viable Product (MVP)** is to deliver the core operational backbone of the Explorer Universe Production System (EUPS). Rather than attempting to deploy full enterprise extras (such as 3D WebGL spatial viewers or multi-agent autonomous swarm simulations), the MVP focuses strictly on a **constitutionally governed, AI-assisted living universe management core**.

CORE MVP OBJECTIVES

- Governance & Canon Locking:** Ingestion of studio documents/bibles, cryptographic SHA-256 locking, and database write-protection triggers.
- Dynamic Creation & Ingestion Engine:** Ingestion of existing 30+ PDFs via Document Ingestion Engine, dynamic JSON Schema entity registration, and point-in-time version restores.
- Hybrid Human + AI Pipeline:** AI candidate generation in isolated tables, 7-stage gate review workflow, and human-exclusive write authority.

11 CORE MVP MODULE SPECIFICATIONS

Module 1: Application Foundation

Purpose: Base runtime infrastructure, monorepo setup, database connectivity, container orchestration, and system health monitoring.

DB Tables: `eups_system_config`, `eups_audit_log` | **Frontend Views:** App Root Shell (`/app/layout.tsx`), Navigation Sidebar

Backend Services: `CoreConfigService`, `HealthCheckController` (`GET /api/v1/health`), `DatabaseModule`

User Actions: Select theme, view system status | **AI Actions:** System connectivity self-check reporting

Completion Criteria: Docker Compose spins up PostgreSQL 16, Redis 7, Qdrant, Next.js/NestJS; health check API returns HTTP 200.

Module 2: User Authentication & RBAC

Purpose: User identity management, JWT session tokens, user roles, and access control guards.

DB Tables: `eups_user`, `eups_user_session` | **Frontend Views:** Login View (`/auth/login`), Profile Inspector (`/auth/profile`)

Backend Services: `AuthService` (bcrypt + JWT), `JwtAuthGuard`, `RolesGuard`

User Actions: Log in/out, view active assigned Role (`ADMIN`, `APPROVER`, `REVIEWER`, `WRITER`, `CREATOR`) | **AI Actions:** None (Security isolation)

Completion Criteria: Unauthenticated requests return HTTP 401; role guards block unauthorized access.

Module 3: Main Dashboard

Purpose: Central command console displaying system health, gate queues, document counts, import jobs, and recent canon locks.

DB Tables: Reads `eups_universal_entity`, `eups_document_import_job`, `eups_ai_output`, `eups_audit_log` | **Frontend Views:** Studio Dashboard (`/dashboard`)

Backend Services: `DashboardAggregatorService` (GET `/api/v1/dashboard/metrics`)

User Actions: Filter metrics by category, click queue items to jump to workspace | **AI Actions:** Provide live activity summaries

Completion Criteria: Dashboard renders counts and queue metrics in under 200ms.

Module 4: Document & Bible Management

Purpose: Managing creation, editing, viewing, and markdown compilation of studio standards and in-world Bibles (`SB-001` to `SB-004`).

DB Tables: `eups_document` | **Frontend Views:** Document Library (`/documents`), Document Editor (`/documents/[id]`)

Backend Services: `DocumentService`, `MarkdownCompilerService`

User Actions: Create/edit bibles, view clause trees and authority rankings | **AI Actions:** Auto-tag referenced Document Codes (`DC-001`, `SC-001`)

Completion Criteria: Full text editing, search, and clause tagging functional for studio documents.

Module 5: Authority Record & Locking Engine

Purpose: SHA-256 authority locking, digital signature logs, and DB write-block triggers preventing canon overwrites.

DB Tables: `eups_authority_record`, DB Trigger: `trg_protect_locked_canon` | **Frontend Views:** Authority Lock Modal, Lock Ledger Drawer

Backend Services: `AuthorityLockService` (Generates SHA-256 hashes, sets `is_locked = TRUE`)

User Actions: Authorize formal lock on canonical entity, view verification hash | **AI Actions:** None (Human-exclusive lock authority)

Completion Criteria: Direct SQL `UPDATE` or `DELETE` queries against locked rows throw PostgreSQL exceptions.

Module 6: Dynamic Entity Registry

Purpose: Enables system expansion by storing definitions and dynamic JSON Schemas for all entity categories (`Species`, `Technology`, etc.).

DB Tables: `eups_entity_type_registry` | **Frontend Views:** System Expansion Panel (`/admin/registry`), Schema Builder

Backend Services: `EntityTypeRegistryService` (Registers entity categories, validates JSON schemas)

User Actions: Register entity categories via JSON Schema upload/builder | **AI Actions:** Validate JSON Schema syntax and recommend attribute constraints

Completion Criteria: Registering a new category (e.g., `SPECIES`) exposes form fields without new code deployment.

Module 7: Manual Creation System

Purpose: Dynamic schema-driven Universal Form Builder allowing users to manually create and edit any registered element.

DB Tables: `eups_universal_entity` | **Frontend Views:** Universal Creation Modal (`/entities/create`), Entity Inspector (`/entities/[id]`)

Backend Services: `UniversalEntityService` (Persists polymorphic entities, executes dynamic schema validation)

User Actions: Select category, fill generated input fields, assign authority tier | **AI Actions:** Auto-fill draft suggestions based on human prompt

Completion Criteria: Form Builder correctly renders unique UI inputs driven purely by category JSON Schema.

Module 8: AI Candidate Workspace

Purpose: Unified workspace for running AI candidate generation, managing prompts, inspecting RAG sources, and reviewing AI candidate payloads.

DB Tables: `eups_ai_output` | **Frontend Views:** AI Candidate Panel (`/ai/workspace`), Candidate Inspector Component

Backend Services: `AiOrchestrationService` (Wraps LLM calls, injects vector context), `RAGVectorService` (Qdrant client)

User Actions: Input prompt, trigger candidate generation, accept or discard AI payload | **AI Actions:** RAG vector search, structured candidate generation, contradiction check

Completion Criteria: AI generates payloads written strictly to `eups_ai_output`; production tables remain untouched.

Module 9: Review and Approval Workflow

Purpose: Manages 7-stage candidate lifecycle machine (`CREATION` → `DRAFT` → `REVIEW` → `VALIDATION` → `APPROVAL` → `CANONICAL` → `PRODUCTION`).

DB Tables: `eups_gate_review_log`, updates `eups_universal_entity` | **Frontend Views:** Gate Review Queue (`/review`), Lifecycle Inspector

Backend Services: `LifecycleStateMachineService` (Validates permitted state transitions)

User Actions: Submit elements to review, add decision notes, vote to Approve/Reject | **AI Actions:** Background contradiction check on entry to `VALIDATION`

Completion Criteria: Elements advance sequentially; invalid state jumps or unapproved gate leaps are blocked.

Module 10: Version History & Restore Engine

Purpose: Tracks creation history, delta payloads, and visual red/green diffs, providing point-in-time restore capabilities.

DB Tables: `eups_version_history` | **Frontend Views:** Version Tree & Diff Drawer Component

Backend Services: `VersionControlService` (Calculates JSON diffs, creates snapshots, executes restores)

User Actions: Inspect revision history, view visual red/green diffs, trigger point-in-time restore | **AI Actions:** Auto-generate commit summaries

Completion Criteria: Restoring version 1 creates version 3 with restored payload without deleting historical audit records.

Module 11: Document Ingestion Engine

Purpose: Allows existing 30+ PDFs, Markdowns, and text files to be imported, parsed, classified, vector-indexed, and converted into authority records without re-creation.

DB Tables: `eups_document_import_job`, `eups_document_source`, `eups_document_metadata` | **Frontend Views:** Document Import Wizard (`/documents/import`), Extraction Review Panel (`/documents/import/[job_id]`)

Backend Services: `DocumentParserService`, `MetadataExtractionService`, `AuthorityClassificationService`

User Actions: Upload PDF files, assign category, review AI-extracted clauses, approve import | **AI Actions:** Detect document hierarchy, extract headings/clauses, recommend Authority Tier, chunk & upload vector embeddings to Qdrant

Completion Criteria: User uploads PDF, approves extracted structure, and queries content in AI Workspace via RAG vector search.

EXPLICIT MVP EXCLUSIONS (OUT OF SCOPE)

To guarantee rapid execution, the following enterprise components are explicitly excluded from MVP:

EXCLUDED FEATURE	REASON FOR EXCLUSION	MVP ALTERNATIVE
1. WebGL 3D Spatial Viewer	High visual overhead; non-essential for initial governance.	2D tree views and tabular spatial region inspectors.
2. Multi-Agent AI Swarm	Complex orchestration state; potential prompt collision.	Single unified RAG agent workflow (<code>AiOrchestrationService</code>).
3. Automated USD Engine Exporters	Downstream runtime feature; premature at core setup phase.	JSON manifests and Markdown export packages.
4. Live Acoustic/Physics Engine	High compute requirement; non-critical for asset logging.	Acoustic parameters stored as static metadata attributes.
5. External Studio Multi-Tenancy	Security complexity; internal studio is primary user.	Single-instance internal studio deployment model.

AI CODING AGENT SEQUENTIAL IMPLEMENTATION ROADMAP

AI coding agents must execute the build sequence strictly sprint-by-sprint. Do not move to Sprint N+1 until Sprint N passes 100% of acceptance checks.

SPRINT	TARGET DELIVERABLE	MANDATORY AGENT VERIFICATION CHECK
Sprint 1	Docker & Core DB Infrastructure	<code>npm run test:e2e</code> passes HTTP 200 health check.
Sprint 2	Auth & RBAC Middleware Engine	JWT guards return HTTP 401 on bad tokens; role permissions resolve.
Sprint 3	Document Management, Ingestion & Lock Engine	PDF upload extracts text; SQL <code>UPDATE</code> on locked rows fails via DB trigger.
Sprint 4	Dynamic Entity Registry & Form Builder	Registering new schema dynamically renders valid form inputs in UI.
Sprint 5	Universal Entity Creator & State Machine	Elements progress strictly through 7 states; invalid state leaps are rejected.
Sprint 6	AI Candidate Workspace & RAG Engine	AI retrieves ingested vector context and writes payload ONLY to <code>ai_output</code> .
Sprint 7	Version History & Restore Engine	Restoring version 1 creates version 3 snapshot without mutating audit trail.
Sprint 8	Integration Testing, Dashboard & Launch	100% automated end-to-end integration test pass rate.

TECHNICAL ARCHITECT:

Lead Systems Engineer
Explorer Universe Studio

Digital Signature Verified

BLUEPRINT STATUS:

Approved MVP Scope
Document Code: EUPS-MVP-BLUEPRINT-V1.1

AUTHORIZED FOR SPRINT 1 INITIALIZATION